

Αλγόριθμοι και Πολυπλοκότητα

ΦΡΟΝΤΙΣΤΗΡΙΑΚΕΣ ΑΣΚΗΣΕΙΣ (8)

Άπληστοι Αλγόριθμοι 2/2

21/5/2024

Έφη Μαλέσιου
Διδακτορική φοιτήτρια

Δίνονται n ράβδοι χρυσού διαφορετικού βάρους. Θέλουμε να τις ενώσουμε σε μια. Το κόστος της ένωσης 2 ράβδων χρυσού είναι ίσο με το άθροισμα των βαρών τους. Δώστε έναν αποδοτικό άπληστο αλγόριθμο, που να ελαχιστοποιεί το συνολικό κόστος. Δεν ζητείται απόδειξη ορθότητας. Να υπολογιστεί η πολυπλοκότητα.

Άπληστο κριτήριο: Ένωσε τις 2 ελαφρύτερες ράβδους

- Ταξινομούμε τις ράβδους κατά αύξουσα σειρά βάρους.
- Ενώνουμε τις 2 ελαφρύτερες σε μια.
- Τοποθετούμε τη νέα ράβδο στη σωστή θέση (δηλαδή να είναι όλες σε αύξουσα σειρά βάρους).
- Επαναλαμβάνουμε τη διαδικασία.

Πολυπλοκότητα:

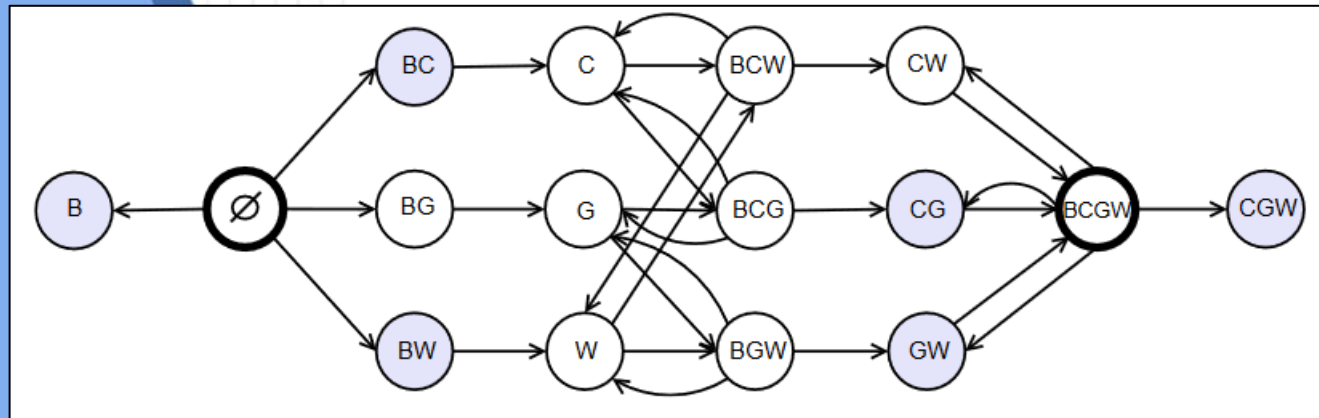
Με χρήση min-heap (παρόμοια δομή με του max-heap, αλλά εδώ ο γονικός κόμβος έχει τιμή μικρότερη από τα παιδιά του): κατασκευή - $O(n)$, και μετά για κάθε βήμα έως την ένωση των n ράβδων: εξαγωγή μικρότερης τιμής $\times 2$ - $O(\log n)$, εισαγωγή νέας τιμής - $O(\log n)$. Άρα, συνολικά είναι: $O(n \log n)$.



Ένας βαρκάρης βρίσκεται στην όχθη ενός ποταμού με ένα λάχανο, μια κατσίκα κι ένα λύκο, και θέλει να τα μεταφέρει στην απέναντι όχθη με τη βάρκα του. Υπάρχουν οι εξής περιορισμοί:

- Για να κινηθεί η βάρκα πρέπει οπωσδήποτε να είναι μέσα ο βαρκάρης.
- Όταν είναι ο βαρκάρης μέσα στη βάρκα, υπάρχει χώρος μόνο για ένα ακόμα αντικείμενο.
- Αν ο λύκος μείνει μόνος με την κατσίκα, θα τη φάει. Ομοίως η κατσίκα θα φάει το λάχανο αν μείνει μόνη μαζί του.

Πώς μπορεί να γίνει η μεταφορά;



Λύση.

Μοντελοποιούμε το πρόβλημα με έναν κατευθυνόμενο γράφο. Κάθε κατάσταση του προβλήματος αντιστοιχεί σε ένα κόμβο το γράφου και οι δυνατές μεταβάσεις μεταξύ των καταστάσεων συμβολίζονται με κατευθυνόμενες ακμές μεταξύ των κόμβων. Ονομάζουμε κάθε κατάσταση ανάλογα με το ποια από τα αντικείμενα που μας ενδιαφέρουν βρίσκονται στην απέναντι όχθη. Συμβολίζουμε:

- με B το βαρκάρη (και τη βάρκα του),
- με C το λάχανο,
- με G την κατσίκα και
- με W το λύκο.

Ο γράφος των καταστάσεων του προβλήματος φαίνεται στο σχήμα . Οι σκιασμένες καταστάσεις δεν επιτρέπονται. Επίσης επιθυμούμε τη συντομότερη δυνατή λύση, άρα υποθέτουμε ότι δε μας ενδιαφέρουν οι λύσεις με κύκλους. Μια ειδική περίπτωση κύκλων είναι οι κύκλοι μήκους 2: υποθέτουμε ότι αν ο βαρκάρης μεταφέρει ένα αντικείμενο από τη μια όχθη στην άλλη, δε μπορεί αμέσως μετά να γυρίσει πίσω με το ίδιο αντικείμενο.

Η λύση του προβλήματος ισοδυναμεί τώρα με την εύρεση ενός μονοπατιού από την αρχική κατάσταση ($\emptyset$) στην τελική ($BCGW$). Η αρχική και η τελική κατάσταση παριστάνονται στο σχήμα με έντονο κύκλο.

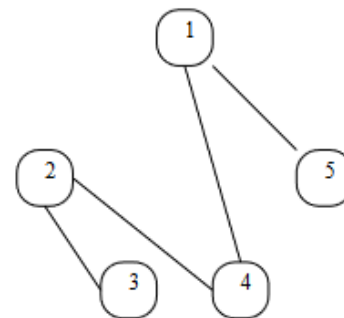
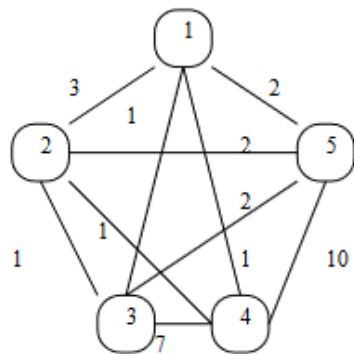
Εύκολα (με DFS/BFS) βλέπουμε ότι η μεταφορά μπορεί να γίνει με δύο τρόπους:

- $\emptyset \rightarrow BG \rightarrow G \rightarrow BCG \rightarrow C \rightarrow BCW \rightarrow CW \rightarrow BCGW$ και
- $\emptyset \rightarrow BG \rightarrow G \rightarrow BGW \rightarrow W \rightarrow BCW \rightarrow CW \rightarrow BCGW$.



Δίνεται ένας πλήρης γράφος $K_5 = (V, E, W)$, $|V| = 5$ το πλήθος των κόμβων, $|E| = m$, το πλήθος των ακμών και $W : E \rightarrow \mathcal{N}$ μία συνάρτηση που αναθέτει βάρη στις πλευρές. Απαριθμήστε τις πλευρές αυθαίρετα $e_1, e_2, \dots, e_m$. Τα βάρη για τις πλευρές e_1, e_3 και e_m είναι αντίστοιχα $W(e_1) = 3$, $W(e_3) = 7$ και $W(e_m) = 10$. Τα υπόλοιπα βάρη επιλέγονται αυθαίρετα στο σύνολο $\{1, 2\}$.

1. Δώστε έναν άπληστο αλγόριθμο σε ψευδογλώσσα που βρίσκει μια εφικτή λύση του προβλήματος του πλανόδιου πωλητή. Υπολογίστε την πολυπλοκότητα του αλγορίθμου σας όταν ο γράφος είναι τάξης n .



Πλήρης γράφος $G = (V, E, W)$ Δένδρο επικάλυψης ελαχίστου κόστους από τον πλήρη γράφο $G = (V, E, W)$

Λύση.

1. Ένας άπληστος αλγόριθμος που λύνει το πρόβλημα του πλανόδιου πωλητή, χρησιμοποιεί τον αλγόριθμο του Prim για να βρει ένα δέντρο επικάλυψης ελαχίστου κόστους. Στη συνέχεια κάνει μια preorder διάσχιση του δένδρου και βάζει τα στοιχεία σε μια λίστα. Τέλος, κάνει έναν κύκλο Hamilton για να επισκεφθεί τους κόμβους που είναι στη λίστα που μπήκαν οι κόμβοι από την preorder διάσχιση δένδρου. Η preorder διάσχιση δένδρου αναδρομικά επισκέπτεται κάθε κόμβο ενός δέντρου, εισάγοντας το σε μια λίστα όταν εμφανίζεται πρώτη φορά, δεν έχει ήδη εισαχθεί. Τα παραπάνω δίνουν τον Αλγόριθμο 1.

Η πολυπλοκότητα του αλγορίθμου είναι ίση με την πολυπλοκότητα του αλγορίθμου Prim, που είναι $O(n^2)$. Επιπλέον χρειάζεται $O(n)$ για την preorder διάσχιση του δένδρου. Επομένως, θέλει συνολικά $O(n^2)$ χρόνο.

Αλγόριθμος 1 GREEDY-TSP(G, c)

- 1: select a vertex $r \in V[G]$ to be a “root” vertex.
 - 2: compute a minimum spanning tree T for G from root r using MTS-Prim(G, c, r).
 - 3: let L be the list of vertices visited in a preorder tree walk of T
 - 4: **return** the Hamiltonian cycle H that visits the vertices in the order L
-

Θέλουμε να επιλέξουμε τον τρόπο παροχής ίντερνετ στο σπίτι, με μηνιαίο πακέτο (χωρίς δέσμευση, μεταβλητή τιμή), ετήσιο συμβόλαιο (δέσμευση, σταθερή τιμή) ή κάποιον συνδυασμό τους σε ορίζοντα n μηνών, π.χ. για 4 έτη

Ποιος είναι ένας άπληστος αλγόριθμος για αυτό το πρόβλημα;

Ένας απλός άπληστος αλγόριθμος δέχεται ως είσοδο τις τιμές $C, p_1, p_2, \dots, p_n$. Τον μήνα d_n ελέγχεται αν το συνολικό κόστος αγοράς μηνιαίου πακέτου για τον τρέχοντα μήνα και καθένα από τους επόμενους 11 είναι μικρότερο από C . Αν αυτό ισχύει, τον μήνα m_i επιλέγεται η αγορά μηνιαίου πακέτου σε τιμή p_i , διαφορετικά επιλέγεται το ετήσιο συμβόλαιο με τιμή C και συνεχίζει στον μήνα m_{i+12} .

```
1: procedure BUYHOMEINTERNET( $C, p_1, p_2, \dots, p_n, n$ )
2:    $cost \leftarrow 0$ 
3:    $i \leftarrow 1$ 
4:   while  $i \leq n$  do
5:     if  $(i + 11) > n$  or  $C > \sum_{k=i}^{i+11} p_k$  then
6:       ▷ Επίλεξε το μηνιαίο πακέτο
7:        $cost \leftarrow cost + p_i$ 
8:        $i \leftarrow i + 1$ 
9:     else
10:      ▷ Επίλεξε το ετήσιο συμβόλαιο
11:       $cost \leftarrow cost + C$ 
12:       $i \leftarrow i + 12$ 
13:    end if
14:  end while
15:  return cost
16: end procedure
```

Ωστόσο, μπορεί να διαπιστωθεί με ένα αντιπαράδειγμα, ότι δεν οδηγεί πάντοτε στη βέλτιστη λύση, καθώς για $n = 13, C = 12$ και $p = \{1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1000\}$, θα επιλέξει αρχικά 1 ετήσιο συμβόλαιο και μετά το μηνιαίο πακέτο, οδηγώντας σε συνολικό κόστος 1012 ενώ θα μπορούσε να επιλεγεί αρχικά 1 μηνιαίο πακέτο και στη συνέχεια 1 ετήσιο συμβόλαιο για τους τελευταίους 12 μήνες, με συνολικό κόστος $13 < 1012$.

2. Έστω ένας πίνακας ακεραίων αριθμών A που αποτελείται από m γραμμές και n στήλες. Θεωρούμε ένα παιχνίδι όπου, ξεκινώντας από οποιοδήποτε κελί της κάτω γραμμής του πίνακα, προσπαθούμε να φτάσουμε σε κάποιο κελί της πάνω γραμμής του πίνακα, περνώντας από κελιά ελαχίστου συνολικού κόστους. Σαν κόστος $A(i, j)$ ενός κελιού (i, j) θεωρούμε τον αριθμό που αναγράφεται στο συγκεκριμένο κελί. Από ένα κελί του πίνακα μπορούμε να κινηθούμε είτε προς το κελί που βρίσκεται ακριβώς επάνω του, είτε διαγνίως επάνω αριστερά, είτε διαγνίως επάνω δεξιά.

Για παράδειγμα, στον πίνακα του παρακάτω σχήματος, από το κελί $(2, 2)$ μπορούμε να κινηθούμε προς κάποιο από τα κελιά $(3, 1)$, $(3, 2)$ ή $(3, 3)$. Σημειώνουμε ότι σαν γραμμή με αριθμό 1 θεωρείται η κάτω γραμμή του πίνακα.

3	9	9	6	9
5	6	7	3	4
6	8	7	7	2
4	3	6	5	9

Τα κελιά με έντονη γραφή δείχνουν τη βέλτιστη διαδρομή στον συγκεκριμένο πίνακα, συνολικού κόστους ίσου με $5 + 2 + 3 + 6 = 16$.

Θεωρήστε τον εξής άπληστο αλγόριθμο επίλυσης του προβλήματος:

- Επιλέγουμε από την πρώτη γραμμή (κάτω) το κελί ελαχίστου κόστους.
- Σε κάθε επανάληψη, από το τρέχον κελί, επιλέγουμε το κελί ελαχίστου κόστους της αμέσως πιο πάνω γραμμής στο οποίο έχουμε το δικαίωμα να μεταβούμε.
- Όταν φτάσουμε στην επάνω γραμμή του πίνακα, ο αλγόριθμος σταματά και υπολογίζει το άθροισμα των κελιών που επισκέφθηκε.

Είναι ο παραπάνω άπληστος αλγόριθμος βέλτιστος; Εάν ναι, να δώσετε απόδειξη ορθότητας του αλγορίθμου. Εάν όχι, να δώσετε ένα αντιπαράδειγμα το οποίο να αποδεικνύει το γεγονός ότι δεν είναι βέλτιστος.

Λύση

Ο άπληστος αλγόριθμος που περιγράφηκε δεν είναι βέλτιστος.

Πράγματι, εάν προσπαθήσουμε να τον εφαρμόσουμε στον πρώτο πίνακα της άσκησης, θα επιλέξουμε αρχικά το κελί με κόστος 3 από την κάτω γραμμή και στη συνέχεια θα επιλεγεί η διαδρομή $(1, 2) \rightarrow (2, 1) \rightarrow (3, 2) \rightarrow (4, 1)$ που είναι συνολικού κόστους 17, ενώ υπάρχει διαδρομή μικρότερου συνολικού κόστους.

Άσκηση 5

3. Το τηλεφωνικό δίκτυο μιας χώρας πρέπει να αναβαθμιστεί για ταχύτερη μεταφορά δεδομένων. Το κόστος αναβάθμισης μιας γραμμής ανάμεσα σε δύο κόμβους είναι ανάλογο του μήκους της. Επειδή για κάθε δύο κόμβους του τηλεφωνικού δικτύου υπάρχουν περισσότερες διαδρομές σύνδεσης (ακολουθία γραμμών) από μία, το κόστος αναβάθμισης είναι μεγάλο.

- i.** Να διατυπώσετε άπληστο αλγόριθμο που θα ελαχιστοποιεί το κόστος αναβάθμισης του δικτύου (για κάθε δύο κόμβους θα υπάρχει ακριβώς μία αναβαθμισμένη διαδρομή σύνδεσης και το συνολικό κόστος θα είναι το ελάχιστο δυνατό).
- ii.** Δώστε την πολυπλοκότητα χρόνου του αλγορίθμου σας.

Λύση

Το τηλεφωνικό δίκτυο μπορεί να αναπαρασταθεί ως ένα βεβαρημένο γράφημα $G = (V, E)$. Άρα, το πρόβλημα ελαχιστοποίησης του κόστους αναβάθμισης του δικτύου ανάγεται στην εύρεση του Minimum Spanning Tree για το γράφημα G . Μπορούμε να χρησιμοποιήσουμε τους γνωστούς αλγορίθμους Kruskal ή Prim.

6. Περιγράψτε έναν αποδοτικό αλγόριθμο ο οποίος, δεδομένου ενός συνόλου σημείων $\{x_1, x_2, \dots, x_n\}$ τα οποία ανήκουν στον άξονα των πραγματικών αριθμών, καθορίζει το μικρότερο σύνολο κλειστών διαστημάτων μοναδιαίου μήκους το οποίο εμπεριέχει όλα τα δοθέντα σημεία.

Λύση

Θεωρήστε τον ακόλουθο πολύ απλό, άπληστο (greedy) αλγόριθμο:

Αρχικά, ταξινομήστε τα δοθέντα σημεία (κατά αύξουσα ή φθίνουσα σειρά) και θα προκύψει η νέα ακολουθία $\{y_1, y_2, \dots, y_n\}$. Το πρώτο διάστημα είναι το $[y_1, y_1 + 1]$. Αν y_i είναι το αριστερότερο σημείο που δεν περιέχεται σε κανένα υπάρχον διάστημα, τότε το επόμενο διάστημα είναι το $[y_i, y_i + 1]$ κ.ο.κ..

5. Το Τμήμα Πληροφορικής ενός Πανεπιστημίου θέλει να κατανείμει ένα σύνολο m μαθημάτων στις διαθέσιμες αίθουσες, οι οποίες είναι n το πλήθος, όπου n αρκετά μεγάλο. Οποιοδήποτε μάθημα μπορεί να λάβει χώρα σε οποιαδήποτε αίθουσα, αρκεί να μην υπάρχουν χρονικές επικαλύψεις μεταξύ μαθημάτων που εξυπηρετούνται από την ίδια αίθουσα. Θέλουμε να προγραμματίσουμε όλα τα μαθήματα, χρησιμοποιώντας όσο το δυνατόν λιγότερες αίθουσες. Δώστε έναν αποδοτικό άπληστο (greedy) αλγόριθμο ο οποίος να αποφασίζει ποια αίθουσα θα φιλοξενήσει ποιο μάθημα. Δίνει ο αλγόριθμός σας βέλτιστο αποτέλεσμα; Εξηγήστε.

Λύση

Διατηρούμε ένα σύνολο F από ελεύθερες αίθουσες, που χρησιμοποιήθηκαν στο παρελθόν και πλέον έχουν αποδεσμευτεί, καθώς κι ένα σύνολο B από κατειλημμένες αίθουσες. Ταξινομούμε τα μαθήματα κατά αύξουσα ώρα έναρξης. Όταν φτάσει η ώρα να ξεκινήσει το επόμενο μάθημα, αφαιρούμε μια αίθουσα από το σύνολο F , ορίζουμε το μάθημα να γίνει σε αυτήν την αίθουσα και την προσθέτουμε στο σύνολο B . Εάν το σύνολο F είναι κενό, προσθέτουμε μια νέα, αχρησιμοποίητη αίθουσα στο σύνολο αυτό. Όταν ένα μάθημα τελειώσει, αφαιρούμε την αίθουσα από το σύνολο B και την προσθέτουμε στο F .

Ο αλγόριθμός μας παράγει βέλτιστο αποτέλεσμα, διότι κοιτάζει να ανακυκλώνει ήδη χρησιμοποιημένες αίθουσες και μόνο αν είναι απολύτως απαραίτητο (δηλαδή αν κάποια στιγμή όλες οι αίθουσες που έχει εξειτάσει είναι κατειλημμένες και προκύπτει ζήτηση για μία ακόμη) επιλέγει μια νέα, αχρησιμοποίητη αίθουσα. Σε καμία περίπτωση δε χρησιμοποιεί παραπάνω από m αίθουσες, ενώ στη χειρότερη περίπτωση, όπου όλα τα μαθήματα διεξάγονται ταυτόχρονα, θα χρησιμοποιήσει ακριβώς m αίθουσες.

8. Η Alice θέλει να διοργανώσει ένα πάρτι και σκέφτεται ποιους να καλέσει. Έχει n άτομα από τα οποία θα επιλέξει και έχει καταρτίσει και μια λίστα με τα ζεύγη ατόμων που ο ένας γνωρίζει τον άλλον. Θέλει να διαλέξει όσο το δυνατόν περισσότερα άτομα, με δύο μόνο περιορισμούς:

- (α') Στο πάρτι, κάθε άτομο θα πρέπει να γνωρίζει τουλάχιστον άλλα πέντε άτομα και
- (β') να υπάρχουν πέντε άλλα άτομα τα οποία δε θα γνωρίζει.

Δώστε έναν αποδοτικό αλγόριθμο ο οποίος δέχεται σαν είσοδο τη λίστα των n υποψήφιων προσκεκλημένων και τη λίστα με τα ζεύγη που ο ένας γνωρίζει τον άλλον και να παράγει στην έξοδο την καλύτερη επιλογή για τους προσκεκλημένους στο πάρτι. Δώστε το χρόνο εκτέλεσης συναρτήσει του n .

Λύση

Η ιδέα εδώ είναι να εντοπίσουμε τα άτομα που δεν μπορούν να παρευρεθούν και να τα αφαιρέσουμε από τη λίστα των προσκεκλημένων. Θα κατασκευάσουμε έναν άπληστο αλγόριθμο παρόμοιο με αυτόν που φαίνεται παρακάτω (Αλγόριθμος 1).

Μπορούμε να θεωρήσουμε ότι οι φίλοι της Alice σχηματίζουν ένα μη-κατευθυνόμενο γράφο με τόσες κορυφές όσοι είναι οι φίλοι και τόσες ακμές όσες είναι οι εγγραφές της λίστας των γνωριμιών. Στόχος μας είναι να βρούμε και να αφαιρέσουμε από το γράφο όλες τις κορυφές που έχουν βαθμό μικρότερο από 5 ή μεγαλύτερο από $|V| - 5$, μέχρι να μην υπάρχουν άλλες προς αφαίρεση κορυφές (βλ. γραμμές 8, 9, 10).

Σε κάθε επανάληψη, με άπληστο τρόπο αποκλείουμε τους υποψήφιους προσκεκλημένους οι οποίοι δεν πληρούν τα κριτήρια που έχουμε θεσπίσει και, συνεπώς, δεν μπορεί να βρίσκονται στην τελική επιλογή. Η σειρά αποκλεισμού στα πλαίσια μιας επανάληψης δεν εξαρτάται από το πόσους γνωστούς έχει ένας υποψήφιος προσκεκλημένος. Για παράδειγμα, ένας υποψήφιος με 4 γνωστούς και ένας υποψήφιος με μόλις 1 γνωστό θα αποκλειστούν και οι δύο αμέσως μόλις εντοπιστούν κατά τη σάρωση της λίστας των υποψηφίων.

Μπορούμε να διαγράψουμε αυτούς τους υποψηφίους με ασφάλεια, καθώς αν κάποιος γνωρίζει λιγότερους από 5 ανθρώπους, θα εξακολουθεί για πάντα να έχει αυτή την ιδιότητα, όσοι και αν διαγραφούν από τη λίστα. Το ίδιο ισχύει και για το άλλο μας κριτήριο: Όσες διαγραφές και να γίνουν από τη λίστα, αν κάποιος γνωρίζει περισσότερους από $|V| - 5$, δε θα χάσει ποτέ αυτή την ιδιότητα. Και στις δύο αυτές περιπτώσεις, λοιπόν, άπληστα διαγράφουμε τους συγκεκριμένους υποψήφιους προσκεκλημένους από τη λίστα. Ο αλγόριθμος θα τερματίσει όταν δεν προκύψουν άλλες αλλαγές στη λίστα των καλεσμένων, δηλαδή όταν δεν υπάρχουν άλλοι υποψήφιοι που να πληρούν τα κριτήρια αποκλεισμού. Επομένως πάντα έχουμε μια ορθή λύση του προβλήματος.

Η πολυπλοκότητα του αλγορίθμου είναι $O(n^3)$, καθώς θα γίνουν το πολύ n περάσματα της λίστας των φίλων και σε κάθε πέραςμα θα υπολογίσουμε το βαθμό των κορυφών σε $O(n^2)$.

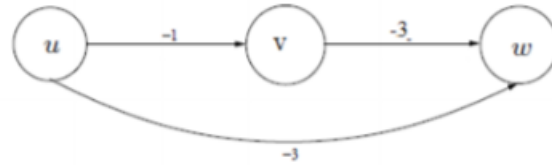
Αλγόριθμος 1 (Άσκηση 8)

```
1: INPUT: Set  $V$  of Alice's friends
2: INPUT: List  $E$  of pairs of acquaintances
3:
4:  $listsUpdated \leftarrow \mathbf{true}$ 
5: while  $listsUpdated = \mathbf{true}$  do
6:    $listsUpdated \leftarrow \mathbf{false}$ 
7:   for  $v \in V$  do
8:     if  $v$  knows less than 5 or more than  $|V| - 5$  then
9:        $V \leftarrow V - \{v\}$ 
10:       $E \leftarrow E - \{acq(v)\}$ 
11:       $listsUpdated \leftarrow \mathbf{true}$ 
12:    end if
13:  end for
14: end while
```

6. (8 Μονάδες) Ο καθηγητής Εξυπνούλης προτείνει τον ακόλουθο αλγόριθμο για την εύρεση της συντομότερης διαδρομής από τον κόμβο s στον κόμβο t σε ένα γράφο με ακμές αρνητικού βάρους: πρόσθεση μιας σταθεράς σε κάθε βάρος ακμής έτσι ώστε όλα τα βάρη να γίνουν θετικά, κατόπιν εκτέλεση του αλγόριθμου του *Dijkstra* για την εύρεση του συντομότερου μονοπατιού από τον κόμβο s στον κόμβο t .

Λειτουργεί τώρα σωστά ο αλγόριθμος;

Λειτουργεί τώρα σωστά ο αλγόριθμος; **Λύση** Λάθος, αντιπαράδειγμα.

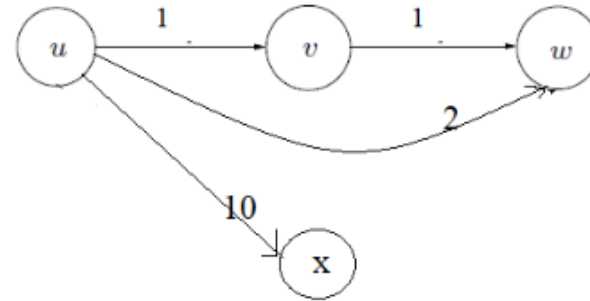


Θα πρέπει να προσθέσουμε 4 στα βάρη των ακμών. Τότε το συντομότερο μονοπάτι από τον u στο w θα είναι η (u, w) , ενώ αρχικά ήταν uvw .

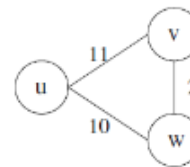
4. (16 Μονάδες) Να αποδείξετε αν είναι σωστές ή λάθος οι παρακάτω εκφράσεις:

- Αν ο γράφος G έχει περισσότερες από $|V| - 1$ ακμές και υπάρχει μια μοναδική ακμή μέγιστου βάρους, τότε αυτή η ακμή δεν μπορεί να είναι τμήμα ενός ΔΕΕΚ.
- Το δέντρο διαδρομών μικρότερου βάρους που υπολογίζεται από τον αλγόριθμο του *Dijkstra* είναι υποχρεωτικά ένα ΔΕΕΚ.
- Έστω γράφος G με διαφορετικά βάρη σε κάθε ακμή. Τότε ο G έχει μοναδικό ΔΕΕΚ.

- Λάθος, αντιπαράδειγμα. Αν η ακμή αυτή είναι γεφυρά αναγκαστικά θα μπει



- Λάθος, αντιπαράδειγμα



Ακμές ΔΕΕΚ (δένδρο επικάλυψης ελάχιστου κόστους):

- (v,w)
- (u,w)

- Σωστό

Ακμές αλγορ. Dijkstra:

- (u,v)
- (u,w)

Λύση.

Θα δείξουμε την εξής ισοδύναμη πρόταση: Ένας γράφος έχει ένα μοναδικό δέντρο επικάλυψης ελάχιστου κόστους (ΔΕΕΚ) T , αν για κάθε διαμέριση των κορυφών του γράφου υπάρχει μία και μοναδική ακμή η οποία δε 'σέβεται' τη διαμέριση (δηλ. τα άκρα της ανήκουν σε διαφορετικά σύνολα της διαμέρισης) και έχει το ελάχιστο βάρος ανάμεσα στις ακμές που δε 'σέβονται' τη διαμέριση.

Θεωρούμε ότι οι κόμβοι που πρέπει να επισκεφθούν μας έρχονται ταξινομημένοι σύμφωνα με το ακόλουθο κριτήριο: οι κόμβοι ταξινομούνται σε αύξουσα σειρά ως προς το ελάχιστο βάρος ακμής που προσπίπτει σε αυτούς. Σε περιπτώσεις ισοπαλίας το κριτήριο εφαρμόζεται αναδρομικά στην επόμενη ακμή ελαχίστου βάρους κ.ο.κ. Θα δείξουμε το ζητούμενο με επαγωγή.

- Αν ο γράφος έχει 1 ή 2 κόμβους, τότε έχει μοναδικό ΔΕΕΚ (προφανές).
- Έστω ότι ο ισχυρισμός μας ισχύει για γράφους με $n = k$ κόμβους. Θα δείξουμε ότι ισχύει για γράφους με $n = k + 1$ κόμβους.

Για $n = k + 1$ κόμβους έχουμε τα ακόλουθα: Τους πρώτους k κόμβους που θα συναντήσουμε μπορούμε να τους ενώσουμε με ένα μοναδικό ΔΕΕΚ (από την επαγωγική υπόθεση). Μένει να δείξουμε ότι ο κόμβος $k + 1$ εισάγεται στο ΔΕΕΚ χρησιμοποιώντας μία μόνο ακμή.

Θεωρούμε τη διαμέριση των κόμβων του γράφου σε δύο σύνολα: το πρώτο περιέχει τους k κόμβους που σχηματίζουν το τρέχον ΔΕΕΚ και το δεύτερο τον κόμβο $k + 1$ (που δεν είναι ακόμη μέσα στο ΔΕΕΚ). Όλες όμως οι ακμές που προσπίπτουν στον $k + 1$ τέμνουν τη διαμέριση (δεν τη 'σέβονται') και έχουν βάρος μεγαλύτερα από κάθε άλλη ακμή που υπάρχει στο τρέχον ΔΕΕΚ (με τους k κόμβους) - διαφορετικά τον κόμβο αυτό θα έπρεπε να τον έχουμε συναντήσει νωρίτερα. Επομένως, για να μην υπάρχει μοναδικό ΔΕΕΚ, θα πρέπει να υπάρχουν τουλάχιστον 2 ΔΕΕΚ T_{k+1} και T'_{k+1} που να συνδέουν το νέο κόμβο $k + 1$ με το ΔΕΕΚ των πρώτων k κόμβων. Αν ισχύει το τελευταίο, τότε τα 2 ΔΕΕΚ θα πρέπει να έχουν το ίδιο συνολικό βάρος (κόστος):

$$W(T_{k+1}) = W(T'_{k+1}),$$

(όπου με $W(T)$ συμβολίζουμε το βάρος του δέντρου T), δηλαδή θα πρέπει να ισχύει

$$W(T_k) + w(x, k + 1) = W(T_k) + w(y, k + 1),$$

(όπου με $w(a, b)$ συμβολίζουμε το βάρος της ακμής (a, b)), οπότε θα πρέπει να υπάρχουν 2 διαφορετικές ακμές οι οποίες να προσπίπτουν στον κόμβο $k + 1$ (έστω x, y τα άλλα άκρα τους αντίστοιχα) και για τις οποίες να ισχύει

$$w(x, k + 1) = w(y, k + 1),$$

δηλαδή να έχουν το ίδιο βάρος! Αυτό είναι άτοπο λόγω υπόθεσης (έχουμε υποθέσει βάρη διαφορετικά ανά δύο), επομένως υπάρχει ένα και μοναδικό ΔΕΕΚ.

Άσκηση 1 (Το 0-1 knapsack πρόβλημα)

Έστω $\{1, \dots, n\}$ αντικείμενα όπου το i -οστό αντικείμενο έχει βάρος $w[i]$ κιλά και αξία $v[i]$. Έχουμε μια τσάντα η οποία μπορεί να σηκώσει βάρος μέχρι W κιλά. Ποια αντικείμενα θα πρέπει να τοποθετήσουμε στην τσάντα για να μεγιστοποιήσουμε την αξία του περιεχομένου της;

Άσκηση 2 (Το κλασματικό knapsack πρόβλημα)

Θεωρήστε παραλλαγή του πιο πάνω προβλήματος όπου μπορούμε να επιλέξουμε ένα κλάσμα ενός αντικειμένου (π.χ. μισό κιλό χρυσόσκονης). Ποια είναι η πιο κερδοφόρα επιλογή αντικειμένων;

- Να λύσετε το πρόβλημα με άπληστο αλγόριθμο.
- Να αποδείξετε την ορθότητα του αλγορίθμου σας και να συζητήσετε τη δυνατότητα ύπαρξης άπληστου αλγορίθμου για το 0-1 knapsack πρόβλημα (Άσκηση 1).

Θέτουμε $a[i]=v[i]/w[i]$ για κάθε αντικείμενο. Υποθέτουμε ότι τα αντικείμενα είναι ταξινομημένα σε φθίνουσα σειρά ως προς την τιμή a . Αν όχι ταξινομούμε με ένα από τους γνωστούς αλγόριθμους ταξινόμησης.

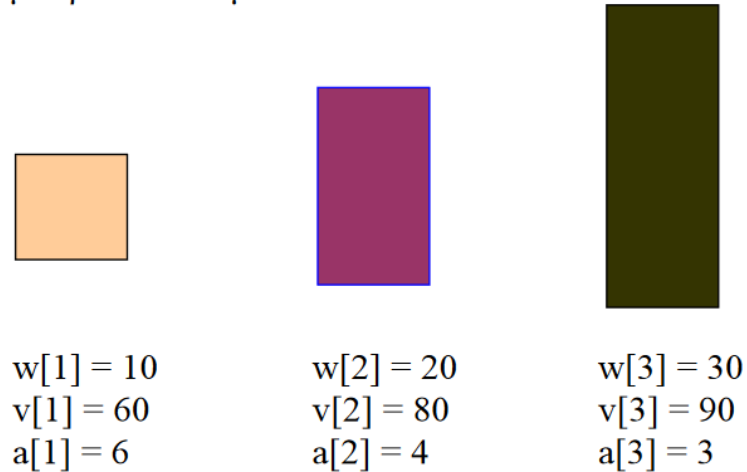
Ακολουθεί άπληστος αλγόριθμος για το πρόβλημα:

```
i =1;
w =0;
A = ∅;
while (i≤n && w!= W){
    if (w+w[i] ≤ W )
        A = A∪{ (i,w[i]) };
        w = w + w[i];
    else
        A = A∪{ (i, W-w) };
        w = W;
        i=i+1;
}
```

Ο αλγόριθμος επιλέγει αντικείμενα βάσει του κριτηρίου ποιο έχει τον μεγαλύτερο λόγο αξίας ανά βάρος. Όταν φθάσει σε κάποιο αντικείμενο του οποίου το βάρος είναι μεγαλύτερο από την υπολειπόμενη χωρητικότητα του σάκου, επιλέγει μόνο μέρος αυτού.

Ακολουθεί αντιπαράδειγμα για την καταλληλότητα του αλγορίθμου στην περίπτωση του προβλήματος 0-1 knapsack:

Θεωρούμε τρία αντικείμενα



και τσάντα με $W = 50$.

Ο άπληστος αλγόριθμος επιλέγει τα δύο πρώτα αντικείμενα.
Συνολικό κέρδος: 140

Η βέλτιστη λύση όμως περιέχει τα δύο τελευταία αντικείμενα.

ΣΥΝΕΧΕΙΑ ΣΤΟ ΕΠΟΜΕΝΟ ΦΡΟΝΤΙΣΤΗΡΙΟ...



Πηγές Φ8:

Σημειώσεις / διαλέξεις μαθήματος, προτεινόμενα βιβλία, υλικό μαθήματος προηγούμενων ετών,
SMA5503, Michael Sambol, Δ. Φωτάκης, Α. Φιλίππου, Μ. Μαυρονικόλας, Simply Explained